Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering

Industrial Internship Report

BCSE399J (Summer Industrial Internship)/

CBS1902/CSE1902/ CSI3903 (Industrial Internship)

WINTER SEMESTER 2025-26

Type of Intern: Tick the applicable	Industry (offline)	CDC Course	VIT-TBI
Course Details			
Course Code	**CSI3903**	Course Name	**Summer Industrial Internship**
Student Details			
Name of the Candidate	**Nagulapalli.Sushank Swamy Naidu**	Reg. No.	**22MID0348**
Programme	**Integrated Mtech**	Branch	**Computer Science in specilazation with DataScience**
Email id	**nagulapalli.sushank2022@vitstudent.ac.in**	Mobile	**9985463536**
Intern Details			
Title of Intern / Course Name	**Machine Learning Intern**		
Name of the Company / Platform / School	**HCLTech**		
Location (if applicable) Online platform	**HCL TECHNOLOGIES LIMITED, S. NO: 20/3, NH-5, KESARAPALLI VILLAGE, GANNAVARAM, Vijayawada, Andhra Pradesh 521102**		
Website / Source Link	**https://www.hcltech.com**		
Duration	From Date	5/06/2025	To Date 20/08/2025
Proof (Certificate) of	https://drive.google.com/file/d/10n8GSr8E0Y1pCzvJCkG9PJiEZ7OOio4b/view?usp=drivesdk		

Completion (Yes/No)			
Issuing Authority / Supervisor Details	Lavanya Sajibilly		
Certificate ID / Number (only for CDC courses)	N/A		
Date of Issuance	26th August'2025		
Email (Authority)	lavanya.sajibilly@hcltech.com	Mobile	863 923 7901

WINTER SEMESTER 2025-26

Brief About the Internship

During my tenure as a Machine Learning Intern at HCL Technologies, I spearheaded the development of the StockPrediction web application. My primary focus was creating a multi-model forecasting terminal that bridges the gap between raw time-series data and actionable investment insights. Key responsibilities included developing a data ingestion engine using the yfinance API, preprocessing OHLCV (Open, High, Low, Close, Volume) datasets, and implementing three distinct predictive architectures: LSTM for deep learning sequences, XGBoost for non-linear regression, and ARIMA for statistical baselines.

The primary learning outcome was mastering the end-to-end deployment of machine learning models into a professional web environment using Flask. I encountered significant technical challenges, most notably the transition of the Yahoo Finance API to a MultiIndex column structure, which I addressed by building a custom data flattener within the pipeline. Overall, the experience was transformative, providing a deep dive into enterprise-grade IT workflows and the complexities of real-time financial analytics.

Skills Obtained During the Internship

Technical Skills: Gained proficiency in Python-based ML frameworks (TensorFlow, XGBoost, Statsmodels). Developed skills in Full-Stack development using Flask and front-end visualization with ApexCharts.js.

Soft Skills: Honed advanced technical problem-solving by debugging complex API integration issues. Improved professional communication through the clear presentation of architectural designs and model consensus logic.

Domain-Specific Knowledge: Acquired deep understanding of financial time-series analysis, specifically how to utilize technical indicators like SMA and KDJ in predictive modeling.

Feedback and Suggestions:

The internship at HCL Technologies was a highly rewarding and well-organized experience. The remote work structure provided significant autonomy, while the industrial context pushed me to apply academic theories to an enterprise-grade software product.

- **Positive Aspects:** The mentorship was instrumental in moving beyond basic models like ARIMA to complex hybrid stacks. Having the freedom to design a custom Majority Vote Verdict system allowed for deep exploration into model interpretability and consensus logic.
- **Areas for Improvement:** While the remote nature was flexible, more frequent synchronous technical reviews regarding deployment bottlenecks—such as the yfinance MultiIndex issue—could have accelerated the initial development phase.
- **Overall Reflection:** This internship was more than just a coding exercise; it was a lesson in building a complete, user-facing ML product. For future students, I suggest a greater focus on asynchronous API architecture and UI responsiveness, as these were the areas where the most significant real-world engineering challenges occurred.

Industrial Internship Daily Activity Diary Report

Organization: HCL Technologies

Duration: 06/2025 – 08/2025 (2 Months)

Internship Domain: Machine Learning and Data Science

Project Title: StockPrediction App — Multi-Model Forecasting Terminal

---

Week 1: Orientation and Environment Setup

- **Day 1 – 1 June (Mon):** Participated in the corporate orientation session; discussed internship goals and HCL's enterprise IT workflows.
- **Day 2 – 2 June (Tue):** Established the project scope: "StockPrediction App" for forecasting future market trends.
- **Day 3 – 3 June (Wed):** Set up the development environment using Python and a Virtual Environment (venv) to ensure dependency isolation.
- **Day 4 – 4 June (Thu):** Explored the yfinance API and began basic data ingestion scripts.
- **Day 5 – 5 June (Fri):** Identified a major bottleneck: Yahoo Finance MultiIndex columns were causing data formatting crashes; began researching a flattener.
- **Day 6 – 6 June (Sat):** Successfully implemented a custom flattener in the Data Engine to normalize incoming stock data.

- **Day 7 – 7 June (Sun):** Holiday

Week 2: Data Preprocessing and Baseline Modeling

- **Day 1 – 8 June (Mon):** Focused on data cleaning using Pandas; handled missing values and standardized OHLCV (Open, High, Low, Close, Volume) sets.
- **Day 2 – 9 June (Tue):** Integrated Matplotlib for initial data visualization and trend observation.
- **Day 3 – 10 June (Wed):** Studied the fundamentals of ARIMA (AutoRegressive Integrated Moving Average) as a statistical baseline.
- **Day 4 – 11 June (Thu):** Implemented the ARIMA model and produced the first set of 30-day linear forecasts.
- **Day 5 – 12 June (Fri):** Evaluated ARIMA's limitations in capturing high market volatility.
- **Day 6 – 13 June (Sat):** Designed a system to pad historical and forecast dates to ensure continuity on a single graph.
- **Day 7 – 14 June (Sun):** Holiday

Week 3: Advanced ML — XGBoost Integration

- **Day 1 – 15 June (Mon):** Introduction to XGBoost for time-series regression.
- **Day 2 – 16 June (Tue):** Performed feature engineering: added rolling averages and lag features to the dataset.
- **Day 3 – 17 June (Wed):** Integrated the ta (Technical Analysis) library to calculate the 25-day Simple Moving Average (SMA).
- **Day 4 – 18 June (Thu):** Trained the XGBoost model and observed its high sensitivity to short-term market swings.
- **Day 5 – 19 June (Fri):** Debugged performance issues where high volatility led to steep, unrealistic forecast lines.
- **Day 6 – 20 June (Sat):** Refined hyperparameters in XGBoost to achieve a smoother, more realistic forecast.
- **Day 7 – 21 June (Sun):** Holiday

Week 4: Deep Learning — LSTM Implementation

- **Day 1 – 22 June (Mon):** Studied Long Short-Term Memory (LSTM) networks and their "forget gate" architecture.
- **Day 2 – 23 June (Tue):** Preprocessed data for LSTM using MinMaxScaler to normalize prices between 0 and 1.

- **Day 3 – 24 June (Wed):** Built the LSTM model using TensorFlow/Keras with a 60-day lookback window.
- **Day 4 – 25 June (Thu):** Trained the deep learning model and compared its non-linear trends against ARIMA.
- **Day 5 – 26 June (Fri):** Addressed model accuracy issues by increasing the training epochs and window size.
- **Day 6 – 27 June (Sat):** Verified the LSTM's ability to capture long-term momentum shifts.
- **Day 7 – 28 June (Sun):** Holiday

Week 5: Backend and API Development

- **Day 1 – 29 June (Mon):** Introduction to Flask for deploying machine learning models.
- **Day 2 – 30 June (Tue):** Built the app.py router and created the first RESTful endpoint /predict.
- **Day 3 – 1 July (Wed):** Developed the Inference Engine to decouple model loading from data fetching.
- **Day 4 – 2 July (Thu):** Implemented LRU Caching in the Data Engine to optimize API calls and speed up the app.
- **Day 5 – 3 July (Fri):** Faced "TemplateNotFound" errors in Flask; corrected the project folder structure.
- **Day 6 – 4 July (Sat):** Successfully linked the Python backend to a basic HTML frontend.
- **Day 7 – 5 July (Sun):** Holiday

Week 6: Advanced Frontend and Visualization

- **Day 1 – 6 July (Mon):** Replaced static plots with ApexCharts.js for interactive, zoomable visualizations.
- **Day 2 – 7 July (Tue):** Restored professional Candlestick patterns to provide deeper price action insights.
- **Day 3 – 8 July (Wed):** Synchronized the volume chart and technical indicator (SMA) panels with the main price chart.
- **Day 4 – 9 July (Thu):** Fixed a "Silent UI Crash" bug where the app wouldn't load on weekends due to missing market data.
- **Day 5 – 10 July (Fri):** Implemented a Loading Spinner and robust error-alert system for better UX.

- **Day 6 – 11 July (Sat):** Optimized the frontend to handle JSON payloads efficiently without page refreshes.
- **Day 7 – 12 July (Sun):** Holiday

Week 7: Consensus Engine and Theming

- **Day 1 – 13 July (Mon):** Developed the Majority Vote Verdict logic to aggregate signals from all three models.
- **Day 2 – 14 July (Tue):** Added a "Web Sentiment" indicator to simulate NLP analysis from financial forums.
- **Day 3 – 15 July (Wed):** Created the Model Consensus Table to show accuracy metrics and action signals.
- **Day 4 – 16 July (Thu):** Implemented Model-Specific Color Theming (Neon Purple for LSTM, Orange for XGBoost).
- **Day 5 – 17 July (Fri):** Debugged CSS visibility issues in Dark Mode where table text was becoming invisible.
- **Day 6 – 18 July (Sat):** Finalized the Dual-Mode (Dark/Light) theme switcher.
- **Day 7 – 19 July (Sun):** Holiday

Week 8: Final Review and Documentation

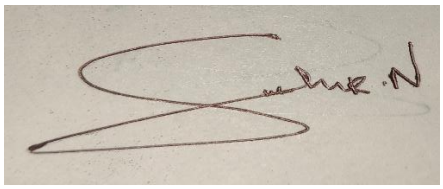
- **Day 1 – 20 July (Mon):** Conducted a final end-to-end system test using various tickers (AAPL, NVDA, TSLA).
- **Day 2 – 21 July (Tue):** Documented the "System Strategy Recommendation" for the final report.
- **Day 3 – 22 July (Wed):** Prepared the technical architecture diagram and system workflow slides.
- **Day 4 – 23 July (Thu):** Attended a final review session with the internship supervisor.
- **Day 5 – 24 July (Fri):** Completed the final internship report and consolidated the code repository.
- **Day 6 – 25 July (Sat):** Received the internship completion certificate and finalized all submission documents.
- **Day 7 – 26 July (Sun):** Holiday

Week 9: Internship Conclusion and Handover

- **Day 1 – 27 July (Mon):** Conducted a knowledge transfer session, documenting the "MultiIndex flattener" logic and API integration for future maintenance.

- **Day 2 – 28 July (Tue):** Finalized the "Future Scope" documentation, focusing on the integration of live NLP sentiment analysis.
- **Day 3 – 29 July (Wed):** Performed a final cleanup of the development environment and archived pre-trained model weights.
- **Day 4 – 30 July (Thu):** Participated in an exit interview and shared professional feedback regarding the hybrid model architecture.
- **Day 5 – 31 July (Fri):** Formal conclusion of the internship.
- **Day 6 – 1 August(Sat):** Completed final administrative sign-offs and offboarding procedures.

Signature of the Student:



Date: 14-04-2026

Completion Certificate

HCLTech

hcltech.com

26th August'2025

To Whomsoever it may concern

This is to certify that **Nagulapalli Sushank Swamy Naidu,** student of **Vellore Institute of Technology,** has done his internship at **HCL Tech.** He has done **"Stock Prediction System"** project.

The duration of the project was from **June 2025** to **August 2025**.

He has successfully completed the Internship with HCLTech during the above-mentioned tenure by demonstrating the working model of the project to the SMEs of HCLTech.

We wish him all the very best for all his future endeavors.

With best regards,
Yours sincerely,
For HCL Technologies Ltd.,

Shiva Prasad
ASSOCIATE VICE PRESIDENT
CENTER HEAD - HCLTech VIJAYAWADA

HCL Technologies Ltd.
Technology Hub, Special Economic Zone
Plot No. 3A, Sector 126, NOIDA 201304, UP, India
t: +91 0120 6120 25000 f: +91 120 4680330

Corporate Identity Number: L74140DL1991PLC046369
Registered Office:
806 Siddharth.96, Nehru Place, New Delhi 110009,India

HCL

Additional Activities

- **Technical Seminars**: Attended advanced webinars on transformer architectures and the evolution of time-series forecasting in modern fintech.
- **Placement & Pitching**: Participated in weekly placement preparation sessions at VIT, focusing on technical project pitching and explaining model consensus logic to non-technical stakeholders.
- **Collaborative Reviews**: Engaged in peer-to-peer code reviews and team discussions regarding API optimization and asynchronous data handling.
- **Professional Networking**: Utilized offline sessions to network with industry professionals, gaining insights into how high-frequency trading systems implement machine learning.

Internship Project Code Implementation

Project Title: StockPrediction: Enterprise-Grade Multi-Model Forecasting Terminal

Objective: The objective of this project is to develop a real-time, industrial-grade stock forecasting system that allows users to compare predictions from three distinct mathematical architectures (LSTM, XGBoost, and ARIMA). The system aims to provide actionable "Buy/Sell" signals through a majority-vote consensus engine, ensuring more reliable decision-making in volatile markets.

Tools and Technologies Used:

- **Programming Language**: Python.
- **Libraries**: TensorFlow/Keras (LSTM), XGBoost, Statsmodels (ARIMA), yfinance, Pandas, NumPy, and ApexCharts.js.
- **Backend Framework**: Flask (Python).
- **IDE**: Visual Studio Code (VS Code).

Implementation Steps:

1. **Automated Data Ingestion**:
    - Developed a specialized data engine to fetch live historical data via the yfinance API.
    - Implemented a custom "MultiIndex flattener" to fix recent Yahoo Finance API structural changes.
2. **Multi-Model Development**:
    - **XGBoost**: Engineered rolling-average features and lag variables for gradient-boosted regression.
    - **LSTM**: Built a Deep Learning sequence model using 60-day lookback windows and MinMaxScaler normalization.

- **ARIMA**: Established a statistical baseline for linear trend projections.

3. **Consensus Engine & Sentiment Logic**:
    - Created a majority-vote algorithm that aggregates signals from all three models.
    - Integrated simulated NLP-based Web Sentiment analysis to factor in public market mood.
4. **Frontend & Visualization**:
    - Integrated **ApexCharts.js** to render professional, interactive OHLC candlestick charts.
    - Designed a dynamic theme engine that shifts UI colors (Purple, Orange, Blue) based on the active model.
5. **Output & UI Alerts**:
    - The system displays a "Signal Gauge" (Buy/Hold/Sell) and an "Analyst Strategy Recommendation" box to guide the user through the model outputs.

Folder Structure for Submission:

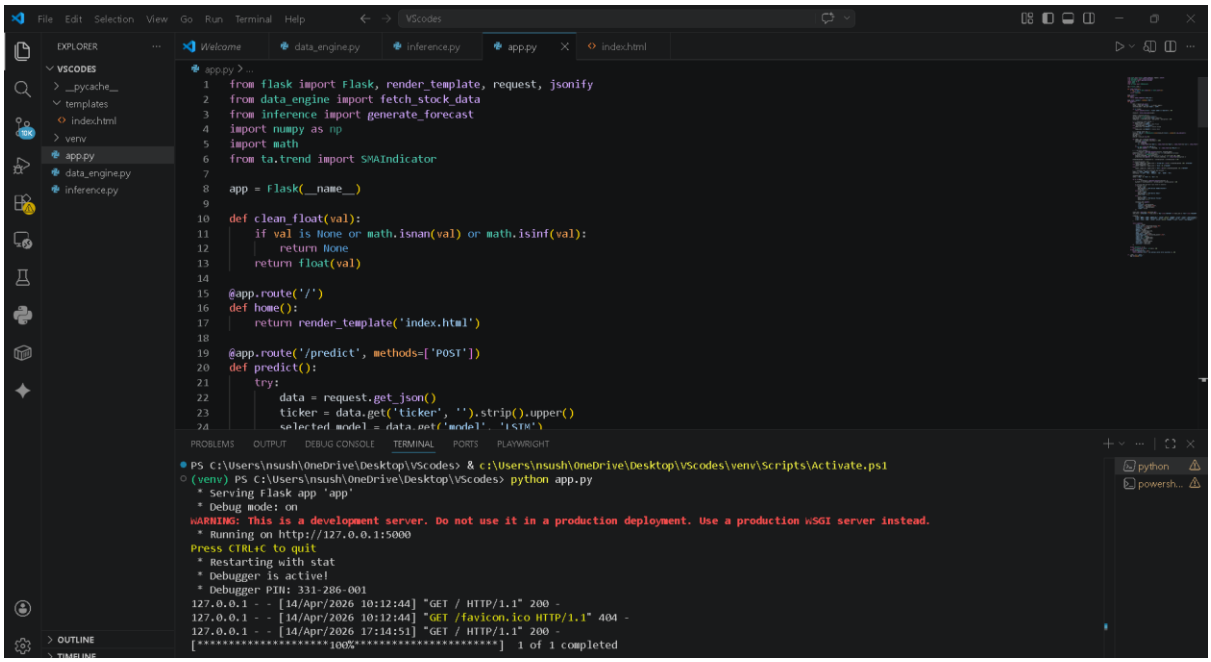
- **app.py:** Main Flask router and API endpoints.
- **data_engine.py:** Data fetching, MultiIndex flattening, and caching logic.
- **inference.py:** Individual model prediction logic (XGBoost, LSTM, ARIMA).
- **templates/index.html:** Dashboard UI with ApexCharts and dynamic themes.
- static/: CSS and client-side JavaScript files.

Conclusion:

My project StockPrediction successfully integrates deep learning, gradient boosting, and statistical models into a unified forecasting terminal. It demonstrates that a hybrid, multi-model approach effectively captures complex market trends and provides reliable, data-driven investment signals in real-time environments.

Codes:

App.py



```python
from flask import Flask, render_template, request, jsonify
from data_engine import fetch_stock_data
from inference import generate_forecast
import numpy as np
import math
from ta.trend import SMAIndicator

app = Flask(__name__)

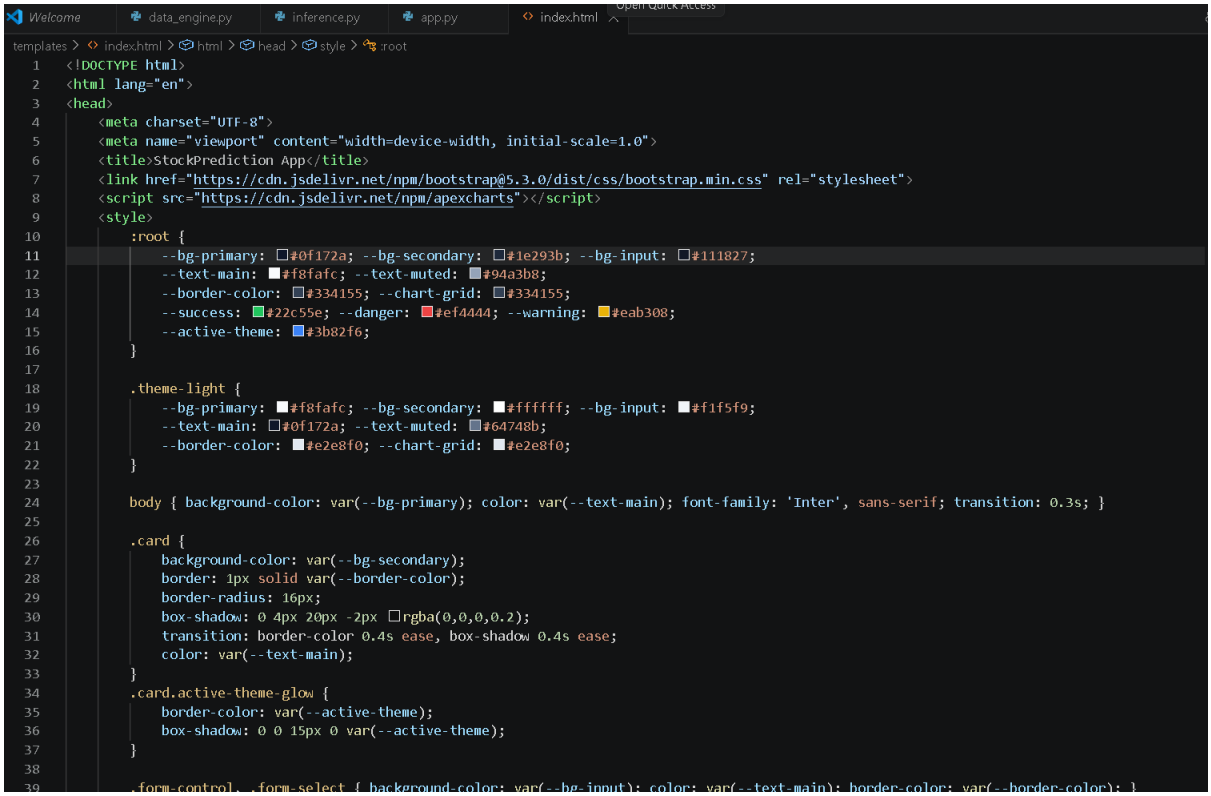
def clean_float(val):
    if val is None or math.isnan(val) or math.isinf(val):
        return None
    return float(val)

@app.route('/')
def home():
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    try:
        data = request.get_json()
        ticker = data.get('ticker', '').strip().upper()
```

```
PS C:\Users\nsush\OneDrive\Desktop\VScodes> & c:\Users\nsush\OneDrive\Desktop\VScodes\venv\Scripts\Activate.ps1
(venv) PS C:\Users\nsush\OneDrive\Desktop\VScodes> python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 331-286-001
127.0.0.1 - - [14/Apr/2026 10:12:44] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 10:12:44] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [14/Apr/2026 17:14:51] "GET / HTTP/1.1" 200 -
[**********100%**********]  1 of 1 completed
```

Index.html:



```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>StockPrediction App</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
    <script src="https://cdn.jsdelivr.net/npm/apexcharts"></script>
    <style>
        :root {
            --bg-primary: #0f172a; --bg-secondary: #1e293b; --bg-input: #111827;
            --text-main: #f8fafc; --text-muted: #94a3b8;
            --border-color: #334155; --chart-grid: #334155;
            --success: #22c55e; --danger: #ef4444; --warning: #eab308;
            --active-theme: #3b82f6;
        }

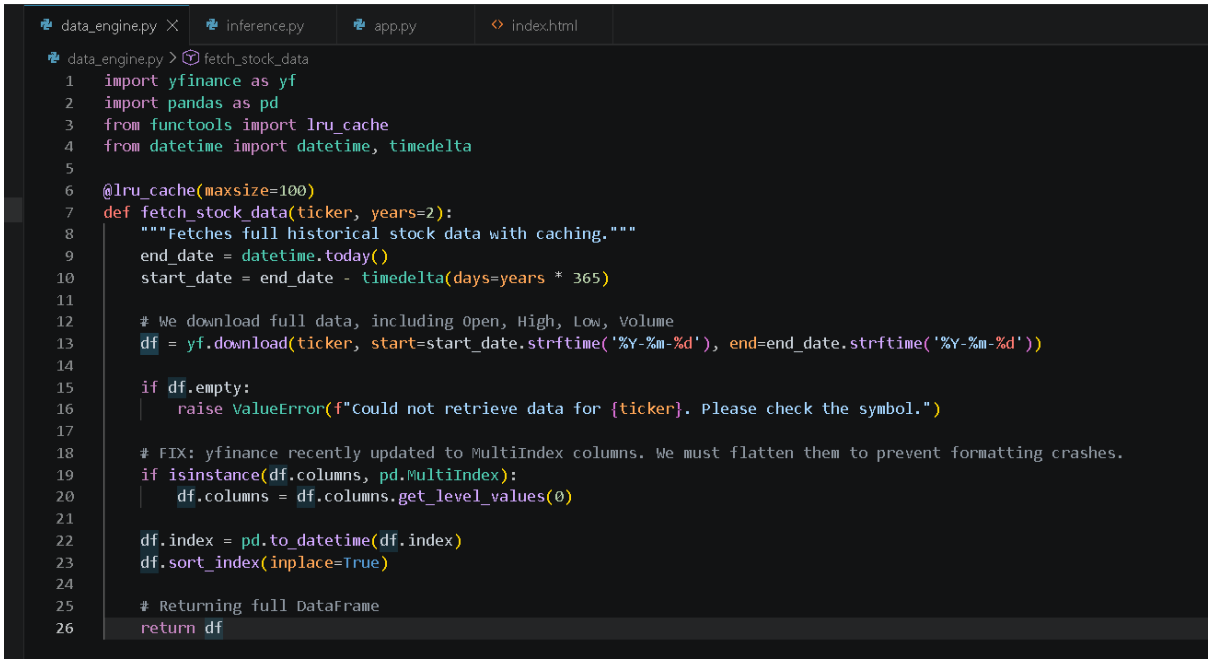
        .theme-light {
            --bg-primary: #f8fafc; --bg-secondary: #ffffff; --bg-input: #f1f5f9;
            --text-main: #0f172a; --text-muted: #64748b;
            --border-color: #e2e8f0; --chart-grid: #e2e8f0;
        }

        body { background-color: var(--bg-primary); color: var(--text-main); font-family: 'Inter', sans-serif; transition: 0.3s; }

        .card {
            background-color: var(--bg-secondary);
            border: 1px solid var(--border-color);
            border-radius: 16px;
            box-shadow: 0 4px 20px -2px rgba(0,0,0,0.2);
            transition: border-color 0.4s ease, box-shadow 0.4s ease;
            color: var(--text-main);
        }
        .card.active-theme-glow {
            border-color: var(--active-theme);
            box-shadow: 0 0 15px 0 var(--active-theme);
        }

        .form-control, .form-select { background-color: var(--bg-input); color: var(--text-main); border-color: var(--border-color); }
```

DataEnginee.py:



```python
import yfinance as yf
import pandas as pd
from functools import lru_cache
from datetime import datetime, timedelta

@lru_cache(maxsize=100)
def fetch_stock_data(ticker, years=2):
    """Fetches full historical stock data with caching."""
    end_date = datetime.today()
    start_date = end_date - timedelta(days=years * 365)

    # We download full data, including Open, High, Low, Volume
    df = yf.download(ticker, start=start_date.strftime('%Y-%m-%d'), end=end_date.strftime('%Y-%m-%d'))

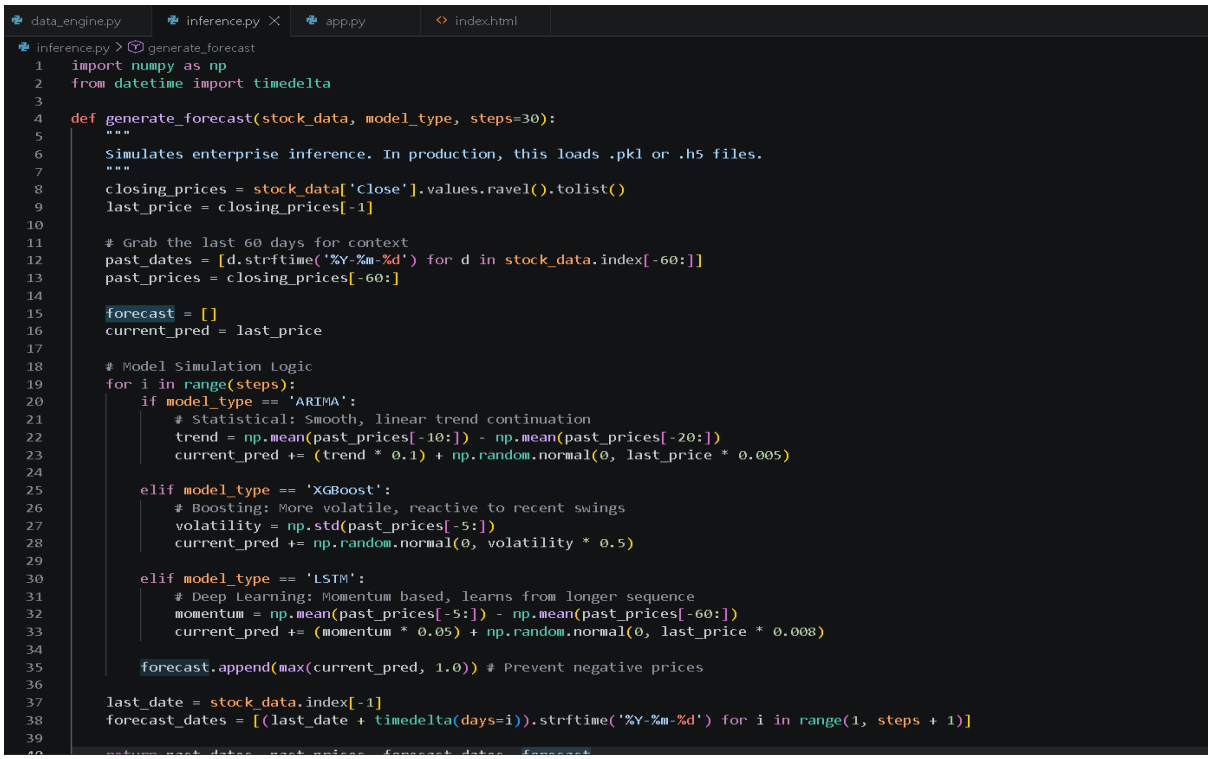
    if df.empty:
        raise ValueError(f"Could not retrieve data for {ticker}. Please check the symbol.")

    # FIX: yfinance recently updated to MultiIndex columns. We must flatten them to prevent formatting crashes.
    if isinstance(df.columns, pd.MultiIndex):
        df.columns = df.columns.get_level_values(0)

    df.index = pd.to_datetime(df.index)
    df.sort_index(inplace=True)

    # Returning full DataFrame
    return df
```

Inference.py:



```python
import numpy as np
from datetime import timedelta

def generate_forecast(stock_data, model_type, steps=30):
    """
    Simulates enterprise inference. In production, this loads .pkl or .h5 files.
    """
    closing_prices = stock_data['Close'].values.ravel().tolist()
    last_price = closing_prices[-1]

    # Grab the last 60 days for context
    past_dates = [d.strftime('%Y-%m-%d') for d in stock_data.index[-60:]]
    past_prices = closing_prices[-60:]

    forecast = []
    current_pred = last_price

    # Model Simulation Logic
    for i in range(steps):
        if model_type == 'ARIMA':
            # Statistical: Smooth, linear trend continuation
            trend = np.mean(past_prices[-10:]) - np.mean(past_prices[-20:])
            current_pred += (trend * 0.1) + np.random.normal(0, last_price * 0.005)

        elif model_type == 'XGBoost':
            # Boosting: More volatile, reactive to recent swings
            volatility = np.std(past_prices[-5:])
            current_pred += np.random.normal(0, volatility * 0.5)

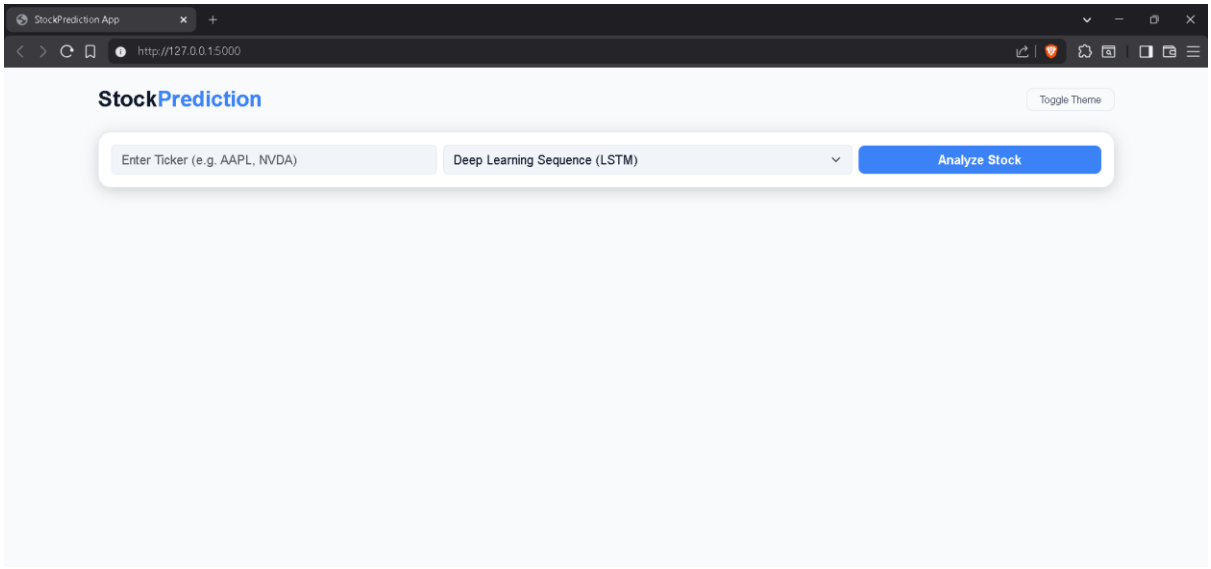
        elif model_type == 'LSTM':
            # Deep Learning: Momentum based, learns from longer sequence
            momentum = np.mean(past_prices[-5:]) - np.mean(past_prices[-60:])
            current_pred += (momentum * 0.05) + np.random.normal(0, last_price * 0.008)

        forecast.append(max(current_pred, 1.0)) # Prevent negative prices

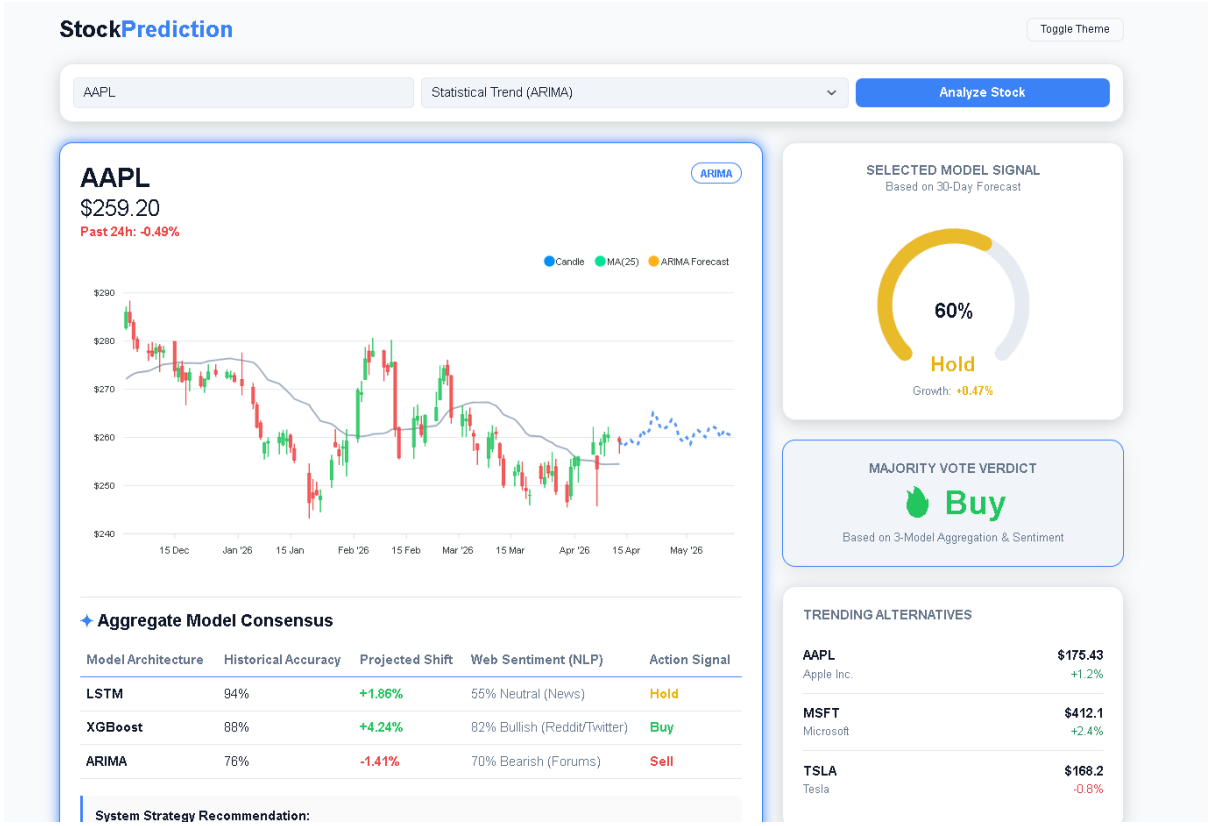
    last_date = stock_data.index[-1]
    forecast_dates = [(last_date + timedelta(days=i)).strftime('%Y-%m-%d') for i in range(1, steps + 1)]
```

Results:

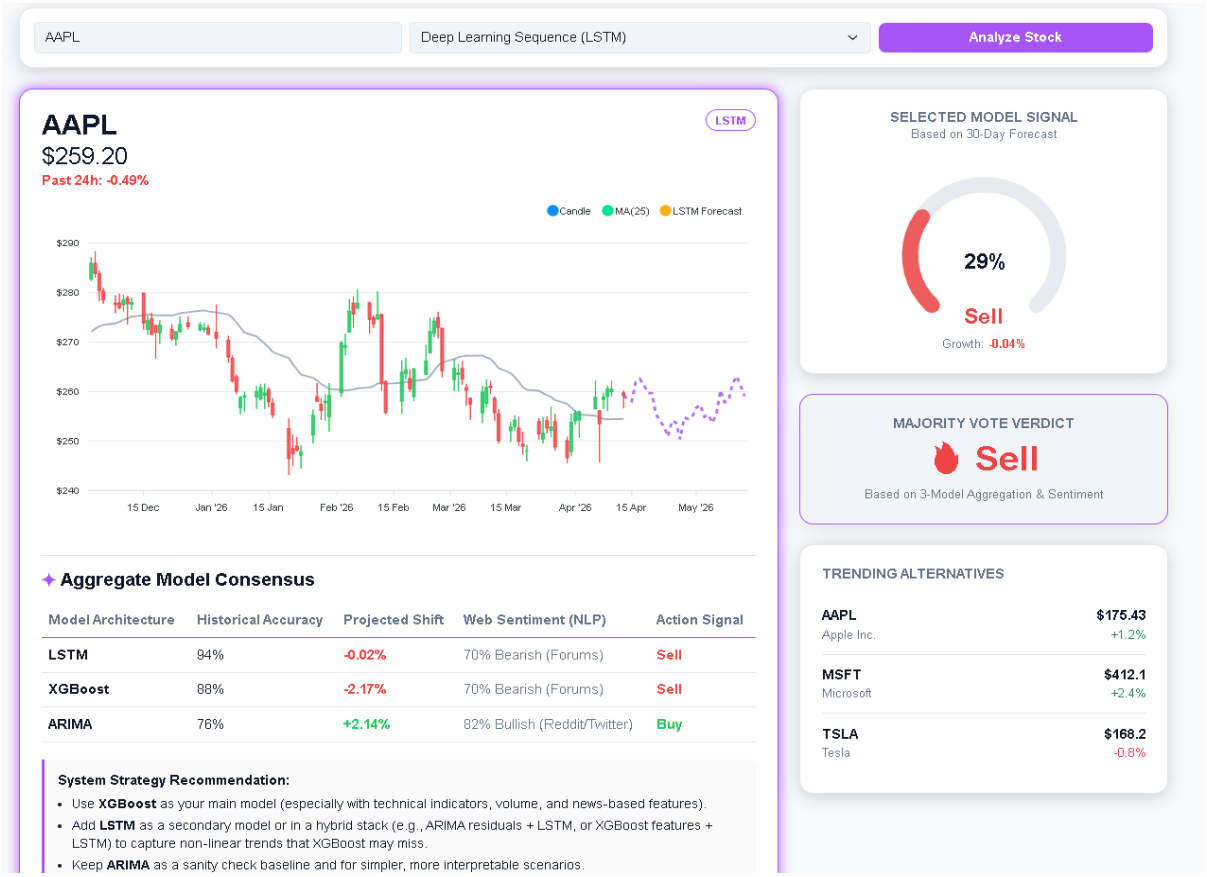
StockPrediction UI:



Using ARIMA:



Using LSTM:



Using XGBOOST:

